

Basic Calculator：一次完整的算法推演记录

题源：Coupang 面经（1p3a）中出现的常规题，等价于 LeetCode 224（带括号）与 227（带乘除）的合并版本。本文记录的是**推导过程本身**，包括三次失败的不变量，以及每次失败暴露出的具体缺陷。

1. 题目

对字符串表达式求值：

- 包含非负整数、`+`、`-`、`*`、`/`、`(`、`)`、空格
- 整数除法向零截断
- 表达式保证合法（不做校验——这是一个应当在面试中**明确说出口**的假设）

样例：`3+2*2` $\rightarrow$ 7；`(1+(4+5+2)-3)+(6+8)` $\rightarrow$ 23；`2*(5+5*2)/3+(6/2+8)` $\rightarrow$ 21

2. 这道题难在哪

难点**不在算法思想**。它有三族标准解法（三层递归下降、单栈+待处理运算符、调度场算法），复杂度都是 $O(n)$ ，没有隐藏的巧思。

难点在**簿记**：要同时管住三件正交的事——

维度	具体要求	典型错法
优先级	<code>*</code> / 高于 <code>+</code> -	递归无条件吞掉右边整段
结合性	同级运算符左结合	<code>8-3-2</code> 算成 7
嵌套	括号可以任意深，且能当操作数	括号单独处理，与操作数逻辑割裂

三者互相牵制：修好优先级往往破坏结合性，反之亦然。下面的推演正是这个过程。

3. 推演过程：三次失败的不变量

3.1 第一版：`dfs(i)` 返回「i 右边全部内容的值」

最自然的第一想法：递归函数返回从当前位置到结尾的整个表达式的值。

测试 `3+2*2`：读 3 $\rightarrow$ 读 `+` $\rightarrow$ 把 `2*2` 整段交给递归拿回 4 $\rightarrow$ `3+4 = 7`。✓

测试 `2*3+4`：读 2 $\rightarrow$ 读 `*` $\rightarrow$ 把 `3+4` 整段交给递归拿回 7 $\rightarrow$ `2*7 = 14`。✗（应为 10）

测试 `8-3-2`：读 8 $\rightarrow$ 读 `-` $\rightarrow$ 把 `3-2` 整段交给递归拿回 1 $\rightarrow$ `8-1 = 7`。✗（应为 3）

诊断：这条不变量让**所有运算符同优先级且右结合**。因为「吞掉右边整段」这个动作对每个运算符都一视同仁，既没有优先级的概念，也没有从左到右结算的机会。

一次失败暴露了两个缺陷，说明它们同源：**递归不知道自己该吃多少**。

3.2 第二版：`*` / 立即折算，`+` - 递归

修法：区别对待两级运算符。遇到 `*` / 时只读一个操作数，当场折进当前值；遇到 `+` - 时才把剩下整段交给递归。

实测结果：

```
ok  2*3+4          -> 10
ok  2+3*4          -> 14
ok  1+2+3          -> 6
ok  2*3*4          -> 24
ok  8/2/2          -> 2
ok  3+2*(5+5)/2    -> 13
ok  1-(5-2)        -> -2
ok  2*(5+5*2)/3+(6/2+8) -> 21
ok  (2+6*3+5-(3*14/7+2)*5)+3 -> -12
XX  3-1-2          -> 4    (want 0)
XX  8-3-2          -> 7    (want 3)
```

优先级修好了，括号也对，但左结合仍然坏的。

诊断： `+` 吞整段没问题（加法满足结合律， $a+(b+c) = (a+b)+c$ ）；`-` 吞整段不行（ $a-(b-c) \neq (a-b)-c$ ）。这是本题第一个关键分岔。

3.3 第三版：取消符号栈，`+` `-` 也立即折算

修法：既然递归导致右结合，那就不递归——`+` `-` 也当场弹栈顶折算。

实测结果：

```
ok  3-1-2          -> 0
ok  8-3-2          -> 3
ok  2*3+4          -> 10
ok  8/2/2          -> 2
ok  1-(5-2)        -> -2
ok  1000/2/2/2/5    -> 25
XX  2+3*4          -> 20    (want 14)
XX  1+2*3+4        -> 13    (want 11)
XX  14-3/2         -> 5     (want 13)
XX  3+2*(5+5)/2    -> 25    (want 13)
XX  2*(5+5*2)/3+(6/2+8) -> 24    (want 21)
XX  (2+6*3+5-(3*14/7+2)*5)+3 -> 108   (want -12)
```

左结合修好了，优先级塌了。典型的按下葫芦浮起瓢。

诊断： `2+3*4` 里，读到 3 时按 `+` 当场算成 5，等读到 `*` 时 3 已经被吃掉了。`+` `-` 的右操作数尚未确定——它可能是 3，也可能是 `3*4*5*6`。所以 `+` `-` 不能立即结算。

4. 三个关键洞察

三次失败之后，问题被压缩成一句话：`*` `/` 必须立即结算，`+` `-` 必须延后结算——延后到什么时候？

洞察一：延后到下一个 `+` `-` 出现

「`1+2+3+4`，最后一个数字来了，前面的加减就可以做了。」

推到底就是：每当遇到一个新的 `+` `-`（或表达式结束、或遇到 `)`），说明前一个加减项的右操作数已经完整读完，可以结算了。

实现上，把待结算的项压栈，末尾统一求和。

洞察二：减号不是运算符，是负号

这是本推演中最省事的一步。

如果 `a-b` 里的 `-` 被理解成「压入 `-b`」，那么：

- 符号栈完全不需要——栈里只存数字
- 顺序无关——`8-3-2` 压入 `[8, -3, -2]`，求和 3，无论从哪端加都对
- 不需要反转、不需要配对

验证：`8-3-2` → 栈 `[8, -3, -2]` → 和为 3。✓

洞察三：括号就是一个操作数

`(1+2)*3` 说明括号能出现在左操作数的位置，`3*(1+2)` 说明也能在右边。所以括号不是独立的语法层，它和数字站在同一个位置：「**读一个操作数**」这件事有两种可能——**数字，或者括号内的整个表达式**，后者的值由同一套逻辑递归求出。

于是递归只用在括号上，`*` / `+` - 全部在同一层循环里靠一个变量分流。

一个附带的顺序技巧

推演中一度反复绕回符号栈，根因是**顺序**：如果先读到运算符再想「记下来待会儿算」，就必然需要暂存。

把顺序倒过来——**先把操作数读到手，再回头看它前面那个运算符是什么**——此刻手上同时有运算符和操作数，一步就能处理完，无需暂存任何符号。

循环的形状因此是「读一个操作数 → 查 `lastop` → 处理」，而不是「读一个符号 → 存起来 → 等操作数」。

5. 最终不变量

函数 `eval()`：

- 前置条件：`p` 指向某个（子）表达式的第一个字符
- 后置条件：`p` 指向闭合它的 `)`，或字符串末尾
- 返回值：该（子）表达式的值

循环内维持的不变量：

1. 栈中只存**数字**，不存运算符
2. 栈中每个元素是一个**已完成的乘除项**，带正负号
3. `lastop` 记录「下一个操作数前面的那个运算符」，初值为 `+`
4. 任意时刻，「已扫描部分的值」== 栈内所有元素之和

第 4 条是核心：它保证末尾直接求和即为答案。

三条处理规则：

lastop	动作	理由
<code>+</code>	<code>push(number)</code>	新的加法项，右操作数待定，延后
<code>-</code>	<code>push(-number)</code>	同上，减号已折进符号
<code>*</code>	<code>pop</code> 栈顶 <code>x</code> ， <code>push(x * number)</code>	优先级最高，立即结算
<code>/</code>	<code>pop</code> 栈顶 <code>x</code> ， <code>push(x / number)</code>	同上，注意 <code>x</code> 是被除数

6. 伪代码

```

eval(s, p):
    st = empty stack of numbers
    lastop = '+'
    while p < len(s):
        skip spaces
        c = s[p]

        if c == ')': break                # 交给调用者消费

        if c is digit or c == '(':
            if c == '(':
                p++                        # 消费 '('
                number = eval(s, p)       # 递归; 返回时 p 停在 ')'
                p++                        # 消费 ')'
            else:
                number = parseNumber(s, p)

            if lastop == '+': st.push(number)
            elif lastop == '-': st.push(-number)
            elif lastop == '*': x = st.pop(); st.push(x * number)
            else:                x = st.pop(); st.push(x / number)
            continue

        if c in "+-*/":
            lastop = c
            p++
            continue

    return sum(st)

```

7. C++ 实现

```

class Calculator {
public:
    int calculate(const string& s) {
        S = &s;
        p = 0;
        return (int)eval();
    }

private:
    const string* S = nullptr;
    size_t p = 0;

    void skipSpaces() { while (p < S->size() && (*S)[p] == ' ') ++p; }

    long long parseNumber() {
        long long x = 0;
        while (p < S->size() && isdigit((unsigned char)(*S)[p])) {
            x = x * 10 + ((*S)[p] - '0');
            ++p;
        }
        return x;
    }
};

```

```

    }

    // 前置 : p 指向子表达式首字符
    // 后置 : p 指向闭合的 ')' 或字符串末尾
    long long eval() {
        vector<long long> st;
        char lastop = '+';
        while (p < S->size()) {
            skipSpaces();
            if (p >= S->size()) break;
            char c = (*S)[p];

            if (c == ')') break;

            if (isdigit((unsigned char)c) || c == '(') {
                long long number;
                if (c == '(') {
                    ++p; // 消费 '('
                    number = eval(); // p 现在停在匹配的 ')'
                    ++p; // 消费 ')'
                } else {
                    number = parseNumber();
                }
                if (lastop == '+') st.push_back(number);
                else if (lastop == '-') st.push_back(-number);
                else if (lastop == '*') { long long x = st.back(); st.pop_back(); st.push_back(x * number); }
                else { long long x = st.back(); st.pop_back(); st.push_back(x / number); }
                continue;
            }

            if (c == '+' || c == '-' || c == '*' || c == '/') { lastop = c; ++p; continue; }
            ++p;
        }

        long long sum = 0;
        for (long long v : st) sum += v;
        return sum;
    }
};

```

位置管理的设计要点（这是推演中卡住最久的地方）：

`p` 是**成员变量**，不是「起点入参+终点出参」。早期版本用 `dfs(s, start, int& end)`，调用方给起点、被调方回填终点，双方需要对账——而 `end` 在不同分支里含义不一致（有时是最后消费的字符位置，有时是下一个待读位置，还夹杂 `+1` 偏移），三种含义混用导致推演无法进行。

改成共享游标后，`end` 这个概念直接消失：每个函数读到哪里就把 `p` 推到哪里，调用方接着往下读即可。**仍然是一遍扫描**，但状态量减半。

8. 复杂度

- **时间**： $O(n)$ ，每个字符最多被访问常数次
- **空间**： $O(n)$ ，栈的深度 + 括号嵌套的递归深度

栈内元素个数等于当前层加减项的数量；递归深度等于括号嵌套层数。

9. 验证结果

手写用例 30 组 + 随机对拍 6000 组，全部通过：

```
hand cases: 30 total, 0 failed
random cases: 6000 total, 0 failed
```

对拍参照实现采用递归下降解析器（expr / term / factor 三层文法），与被测的单栈实现**结构完全独立**，因此能真正验出转移逻辑错误，而不是重复同一个 bug。

手写用例覆盖：

- 单数字、纯空格、首尾空格
- 优先级：3+2*2、2*3+4、1+2*3+4
- 左结合：3-1-2、8-3-2、8/2/2、1000/2/2/2/5
- 除法截断：3/2、1/2、14-3/2
- 括号作为左/右操作数：(1+2)*3、3*(1+2)、(1+2)*(3+4)
- 深度嵌套：(((5))))、100/((2+3)*5)
- 混合长表达式：2*(5+5*2)/3+(6/2+8)、(2+6*3+5-(3*14/7+2)*5)+3

随机生成器构造合法表达式，除数统一改写为 (b*b+1) 以保证非零。

10. 三族解法对比

解法	状态量	代码量	备注
单栈 + lastop（本文）	一个数字栈、一个 lastop、一个游标	最少	递归仅用于括号
三层递归下降	三个函数，每层一个局部值	中等	expr/term/factor，各层不变量独立，最易讲清
调度场算法（双栈）	数字栈 + 符号栈 + 优先级表	最多	通用（支持任意优先级与结合性），面试里不划算

三者渐进复杂度相同，选择依据是**状态量**而非性能。本题失分点全在细节，状态越少越安全。

值得注意：在只有两级优先级的情况下，调度场算法里「比较栈顶与新运算符的优先级」这条规则会退化成「新来的是不是 * / 」这一个判断——不需要优先级表。再往前一步（减号转负号）连符号栈也一并消失。**通用算法在受限场景下的退化，往往就是这道题的最优解。**

11. 复盘

这次推演的有效流程：

1. 说清算法 →
2. 写伪代码直到**完全闭合**（不留「这里要注意」这类占位词）→
3. 翻译成代码 →
4. 手推三步验证不变量

前两步挡住的是算法层的错误，而算法层的错误最贵——语法层的 bug 一编译一运行就现形，算法层的 bug 不做对拍根本看不见，而面试现场没有对拍。

本题的失败模式全部集中在「**不变量定义**」这一层，而不是编码层：

版本	不变量	破在哪
v1	dfs 返回右边全部	优先级 + 结合性同时坏
v2	* / 立即、+ - 递归	结合性
v3	全部立即结算	优先级
v4	* / 立即、+ - 压栈延后	✓

通用教训： 当一个函数同时承担两件不变量不同的事（这里是「吃一串乘除」和「吃一串加减」），无论怎么调都会按下葫芦浮起瓢。出路是把它们分开——要么分成两层函数，要么分成「递归不变量 + 循环不变量」两个层次。本题选了后者。